

INFORME DE TESTING Y VALIDACIÓN DE SOFTWARE

Evaluación Parcial N° 3 - Taller Aplicado de Programación

Proyecto Catch & Go

Sigla	TPY1101
Asignatura	Taller Aplicado de Programación
Entregable	Plan de pruebas, ejecución, evidencias y mejoras del producto
Equipo	Grupo N° 1
Integrantes	Nicolás Iturrieta Villalobos, Marco Tassara Carrasco, Miguel Arredondo Vargas
Jefe de proyecto	Nicolás Iturrieta Villalobos
Analistas de testing	Marco Tassara Carrasco / Miguel Arredondo Vargas
Versión documento	QA 1.0.4
Fecha	18/06/2026

Documento elaborado para evidenciar el plan, ejecución y resultados de pruebas de software del MVP Catch & Go, incluyendo base de datos de prueba, evidencias, mejoras aplicadas, conclusiones y lecciones aprendidas.

Hoja de control del documento

Cliente / usuario objetivo	Público general, empresas y trabajadores temporales de Chile
Producto evaluado	MVP Catch & Go: frontend web, aplicación Android y backend de microservicios
Módulos evaluados	api-gateway, auth-service, profile-service, jobs-service, matching-service, frontend y android-app
Ambiente de pruebas	Ambiente local de desarrollo y QA con ejecución de pruebas unitarias y funcionales
Herramientas utilizadas	JUnit/Gradle, pruebas manuales funcionales, validación visual, planilla de casos de prueba y evidencias en capturas
Criterio de aprobación	Caso aprobado cuando el resultado obtenido coincide con el resultado esperado y no genera errores críticos de seguridad, datos o usabilidad

Registro de cambios

Versión	Cambio	Responsable	Fecha
1.0.0	Versión inicial del plan de pruebas funcionales del MVP	Miguel Arredondo Vargas	08/06/2026
1.0.2	Ajustes de login y validaciones de autenticación	Miguel Arredondo Vargas	15/06/2026
1.0.3	Ajustes generales QA de aplicación Android	Miguel Arredondo Vargas	16/06/2026
1.0.4	Consolidación informe EP3 con evidencias, mejoras y acta de aceptación	Equipo Catch & Go	18/06/2026

Índice

- 1. Introducción y contexto del proyecto
- 2. Alcance, objetivos y criterios de calidad
- 3. Plan de pruebas de software alineado a la problemática
- 4. Base de datos y datos de prueba
- 5. Matriz de trazabilidad de requisitos y casos de prueba
- 6. Casos de prueba y acciones de validación
- 7. Aplicación de pruebas, resultados y evidencias
- 8. Mejoras realizadas según resultados de pruebas
- 9. Cumplimiento de estándares de calidad, ética y seguridad
- 10. Acta de aceptación, conclusión y lecciones aprendidas
- 11. Anexos: evidencias y defensa oral

1. Introducción y contexto del proyecto

Catch & Go es una solución orientada a conectar empresas que requieren personal temporal con trabajadores disponibles para realizar trabajos rápidos de uno o dos días. La problemática principal detectada es la dificultad para encontrar, validar y coordinar trabajadores en poco tiempo, manteniendo información confiable de perfiles, postulaciones, pagos, documentos y ubicación.

En la Evaluación Parcial N° 3 el foco se centra en el aseguramiento de calidad del producto construido. Por lo tanto, este documento reúne el plan de pruebas, la aplicación de pruebas de validación, las evidencias utilizadas, las mejoras incorporadas y la justificación técnica de cumplimiento de calidad, seguridad y ética.

1.1 Resumen de lo desarrollado en EP1 y EP2 orientado al testing

Etapa	Resumen relevante para pruebas
EP1 - Problema y oportunidad	Se definió la necesidad de una plataforma para trabajos rápidos, con roles de trabajador y empresa, registro de usuarios, ofertas, postulaciones y matching.
EP2 - Diseño y construcción	Se construyó una arquitectura con frontend web, app Android y microservicios backend para autenticación, perfiles, ofertas, matching, notificaciones y pagos.
EP3 - Validación	Se validan los componentes desarrollados mediante casos funcionales, unitarios, seguridad JWT, interfaz, base de datos de prueba y evidencias de ejecución.

2. Alcance, objetivos y criterios de calidad

2.1 Alcance

El alcance del testing cubre el MVP del proyecto Catch & Go, incluyendo backend de microservicios Java/Spring Boot, frontend web desarrollado con Next.js/TypeScript y aplicación móvil Android. La validación se enfoca en funcionalidades críticas para el negocio: registro, inicio de sesión, perfiles, publicación de ofertas, postulaciones, matching, dashboard, notificaciones, pagos, seguridad de token y validaciones visuales.

2.2 Objetivos de testing

- Comprobar que las funcionalidades principales respondan correctamente según los requisitos definidos.
- Validar que los distintos componentes del proyecto funcionen de manera integrada y por separado.
- Asegurar buenas prácticas de seguridad, especialmente en autenticación, persistencia de sesión y tokens JWT.
- Detectar desviaciones, registrar evidencias y aplicar mejoras para elevar la calidad del producto final.
- Disponer de documentación clara para la defensa oral y para la aceptación del docente guía.

2.3 Criterios de entrada y salida

Tipo	Criterio	Estado
Entrada	Código base disponible en ambiente de desarrollo/QA y módulos principales ejecutables.	Cumplido
Entrada	Planilla de casos de prueba definida y asociada a módulos del sistema.	Cumplido
Entrada	Datos ficticios de prueba disponibles para usuario, empresa, oferta, postulación y pago.	Cumplido
Salida	Ejecución de casos críticos sin errores bloqueantes.	Cumplido
Salida	Evidencias capturadas y asociadas a resultados obtenidos.	Cumplido

Salida	Mejoras aplicadas y registradas en tabla de control.	Cumplido
--------	--	----------

3. Plan de pruebas de software alineado a la problemática

El plan de pruebas fue diseñado considerando la problemática del proyecto: permitir una contratación temporal rápida, confiable y segura. Por ello, las pruebas se priorizaron en los flujos que podrían afectar directamente la confianza del usuario: autenticación, registro, publicación de ofertas, postulación, matching, pagos y notificaciones.

3.1 Estrategia general de pruebas

Nivel de prueba	Componentes	Objetivo	Técnica aplicada	Resultado esperado
Funcional	Web, Android, microservicios	Validar flujos principales del usuario trabajador y empresa.	Ejecución manual guiada por casos CP.	El flujo cumple con el comportamiento definido y guarda datos correctamente.
Unitaria	Frontend, backend, Android	Aislar métodos, componentes y controladores críticos.	JUnit/Gradle y pruebas de componentes.	Cada unidad retorna el resultado esperado sin depender de otros módulos.
Integración	Gateway, servicios y BD	Comprobar comunicación entre módulos y persistencia.	Flujos con datos de prueba y observación de respuestas.	Los servicios responden y se actualiza la información en la base de datos.
Seguridad	auth-service/JWT	Validar credenciales, expiración y manipulación de token.	Pruebas de token válido, alterado y expirado.	El sistema acepta solo tokens válidos y rechaza accesos indebidos.
Usabilidad	Frontend y Android	Validar mensajes, navegación, botones y retorno.	Checklist de pruebas estándar de QA.	La interfaz comunica correctamente los estados y evita confusión.
Calidad operativa	Notificaciones, pagos, errores	Validar manejo de errores y estados de negocio.	Casos negativos, validaciones y simulación de fallas.	El usuario recibe mensajes claros y no se corrompen datos.

3.2 Funcionalidades a comprobar y resultado esperado

Funcionalidad	Acción de prueba	Módulo asociado	Resultado esperado	Prioridad
Registro de usuarios	Completar formulario con datos válidos e inválidos.	auth-service / frontend / android-app	Se crean usuarios válidos y se bloquean registros incompletos o con formato incorrecto.	Alta
Inicio de sesión	Ingresar credenciales válidas, inválidas y mantener sesión.	auth-service	Se genera token JWT, se ingresa al dashboard correcto y se rechazan credenciales inválidas.	Alta
Perfiles	Actualizar datos de trabajador y empresa.	profile-service	Los cambios se guardan y se muestran correctamente.	Media

Ofertas laborales	Publicar oferta y validar datos de tarifa/ubicación.	jobs-service	La oferta queda activa y los campos inválidos son bloqueados.	Alta
Postulaciones	Postular, listar y gestionar postulantes.	jobs-service	La postulación cambia de estado de forma correcta.	Alta
Matching	Calcular score y ordenar candidatos.	matching-service	El ranking se genera según distancia y habilidades.	Media
Dashboard y navegación	Validar vistas diferenciadas por rol.	frontend web / android-app	Cada rol ve solo las opciones que le corresponden.	Alta
Notificaciones	Validar alertas de cambio de estado y fallas de servicio.	Redis / frontend	Se despliegan mensajes claros en tiempo real o de error amigable.	Media
Pagos	Iniciar y confirmar transacciones, validar campos.	backend / frontend	Se aceptan datos válidos y se rechazan solicitudes incompletas.	Alta
Seguridad JWT	Generar, leer, alterar y expirar token.	backend	El sistema rechaza tokens manipulados o vencidos.	Alta

4. Base de datos y datos de prueba

La base de datos de prueba se utilizó con datos ficticios, evitando información sensible o real de usuarios. Esto permite validar la funcionalidad sin comprometer privacidad ni seguridad. La persistencia considera tablas asociadas a usuarios, roles, perfiles, empresas, ofertas, postulaciones, pagos, documentos y notificaciones.

4.1 Dataset de prueba utilizado

Entidad	Datos de prueba	Uso en pruebas	Control de seguridad/ética
Trabajador	Juan Pérez, juan.perez@email.com, +56912345678, Password123!	Registro, login, perfil, postulación y dashboard trabajador.	Usuario ficticio, sin datos personales reales.
Empresa	Empresa Corp, contacto@empresa.com, +56987654321, CompanyPass!	Registro empresa, publicación de oferta, gestión de candidatos.	Empresa ficticia para evitar uso de datos comerciales reales.
Oferta	Auxiliar de Bodega, Logística, Maipú, \$4.000/hr, ubicación lat/Ing	Publicación, listado, postulación y matching.	Datos de ubicación referenciales y no sensibles.
Postulación	ID oferta 101, ID trabajador 5, estado Pendiente/Aceptado/Rechazado	Validación de cambios de estado y notificaciones.	Estados controlados en ambiente de prueba.
Pago	user-123, monto 10.000, token real-token-123	Inicio y confirmación de pago en controlador.	Token simulado; no se utiliza información bancaria real.
JWT	Subject user@test.com, expiración 3600 segundos, token alterado y expirado	Generación, claims, expiración y tampering.	No se exponen secretos reales en el informe.

4.2 Configuración del ambiente de pruebas

- Ambiente local/QA con ejecución controlada de frontend, backend y aplicación Android.
- Uso de variables de entorno separadas para pruebas, evitando credenciales productivas.
- Ejecución de pruebas unitarias mediante Gradle/JUnit y pruebas funcionales manuales con planilla aprobada.

- Registro de evidencias mediante capturas de pantalla del IDE, consola y resultados de pruebas.
- Revisión de mensajes, navegación, respuestas HTTP y comportamiento esperado frente a errores.

5. Matriz de trazabilidad de requisitos y casos de prueba

La matriz permite demostrar que los casos de prueba cubren los requisitos funcionales relevantes del MVP. La trazabilidad facilita defender qué se probó, con qué propósito y qué parte del producto queda validada.

Requisito	Descripción	Casos de prueba asociados
RF-01	Registro de usuarios trabajador/empresa con validaciones de campos obligatorios, formato de correo y contraseña.	CP-01, CP-02, CP-03, CP-04
RF-02	Inicio de sesión, generación de token JWT, persistencia y control de credenciales inválidas.	CP-05, CP-06, CP-07, CP-08, CP-65, CP-66, CP-67, CP-68
RF-03	Mantenimiento de perfiles, reputación inicial y datos de trabajador/empresa.	CP-09, CP-10, CP-11
RF-04	Publicación de ofertas laborales, validación de tarifa y disponibilidad para trabajadores.	CP-12, CP-13
RF-05	Postulación, seguimiento y gestión de postulantes por parte de empresa.	CP-14, CP-15, CP-16
RF-06	Motor de matching por habilidades, ubicación y visualización de candidatos recomendados.	CP-17, CP-18
RF-07	Dashboards diferenciados por rol, navegación, notificaciones y frontend web.	CP-19, CP-20, CP-21, CP-22, CP-23, CP-24, CP-55, CP-56, CP-57, CP-58, CP-59, CP-60, CP-61, CP-62, CP-63, CP-64
RF-08	Módulos complementarios: pagos, membresías, documentos, seguridad de JWT y app Android.	CP-25, CP-26, CP-27, CP-28, CP-29, CP-30, CP-31, CP-32, CP-33, CP-34, CP-35, CP-36, CP-37, CP-38, CP-39, CP-40, CP-41, CP-42, CP-43, CP-44, CP-45, CP-46, CP-47, CP-48, CP-49, CP-50, CP-51, CP-52, CP-53, CP-54, CP-69, CP-70, CP-71, CP-72

6. Casos de prueba y acciones de validación

A continuación, se consolidan los 72 casos de prueba registrados en la planilla de testing. La tabla resume la acción principal, los datos utilizados, el resultado esperado y el estado de validación obtenido.

6.1 Resumen ejecutivo de ejecución

Total de casos de prueba funcionales/unitarios	72
Casos OK	71
Casos pendientes / sin marca	1
Pruebas estándar QA aplicadas	17
Pruebas estándar OK	14

Pruebas estándar NA	3
Cobertura por componentes	auth-service: 8, profile-service: 3, jobs-service: 5, matching-service: 2, frontend web: 2, Redis / frontend: 2, frontend: 7, android-app: 35, backend: 8

6.2 Tabla completa de casos de prueba

CP	Módulo	Tipo	Funcionalidad a comprobar	Acción a realizar	Resultado esperado / obtenido
CP-01	auth-service	Funcional	Registro de Trabajador con datos correctos	1. Ingresar al formulario de registro en Web o App Android. 2. Ingresar Nombre, Correo, Teléfono, Contraseña, seleccionar rol 'Trabajador'. 3. Presionar el botón 'Registrar'.	Se muestra la pantalla de registro con todos los campos. Los campos se completan correctamente sin errores visuales. Se registra el usuario en BD, se redirige al Login y se muestra un mensaje de éxito. Resultado: OK
CP-02	auth-service	Funcional	Registro de Empresa con datos correctos	1. Ingresar al formulario de registro en Web. 2. Ingresar Nombre, Correo, Teléfono, Contraseña, seleccionar rol 'Empresa'. 3. Presionar el botón 'Registrar'.	Se muestra la pantalla de registro. Los campos se completan correctamente. Registro exitoso de la cuenta de empresa en BD y redirección al login. Resultado: OK
CP-03	auth-service	Funcional	Registro fallido por campos vacíos	1. Ingresar al formulario de registro. 2. Dejar campos en blanco y presionar 'Registrar'.	Se visualiza el formulario. Se muestran alertas visuales indicando que los campos son obligatorios y el registro se bloquea. Resultado: OK
CP-04	auth-service	Funcional	Registro fallido por formato de correo inválido	1. Ingresar al formulario de registro. 2. Ingresar correo sin formato válido (ej: 'correo.com') y completar los demás datos. 3. Presionar 'Registrar'.	Se abre el formulario. El sistema indica que el formato de correo es incorrecto en tiempo real. No se realiza el registro y se mantiene en la misma pantalla. Resultado: OK
CP-05	auth-service	Funcional	Inicio de sesión de Trabajador exitoso	1. Ingresar al Login en la Web o App Android. 2. Ingresar correo y contraseña válidos de Trabajador. 3. Presionar 'Iniciar Sesión'.	Se despliega pantalla de login. Credenciales escritas correctamente. Inicio de sesión exitoso, se genera y guarda el token JWT, y se ingresa al Dashboard de Trabajador. Resultado: OK
CP-06	auth-service	Funcional	Inicio de sesión de Empresa exitoso	1. Ingresar al Login de la Web. 2. Ingresar correo y contraseña de Empresa. 3. Presionar 'Iniciar Sesión'.	Se muestra el login. Datos completados. Ingreso exitoso al Dashboard de Empresa con token JWT activo en cabeceras. Resultado: OK
CP-07	auth-service	Funcional	Inicio de sesión fallido por credenciales inválidas	1. Ingresar al Login. 2. Ingresar contraseña incorrecta o correo no registrado. 3. Presionar 'Iniciar Sesión'.	Se muestra el login. Se ingresan los datos de prueba. Se muestra el mensaje 'Credenciales incorrectas' y se bloquea el acceso. Resultado: OK
CP-08	auth-service	Funcional	Persistencia de sesión (DataStore / LocalStorage)	1. Iniciar sesión correctamente en la App Android o Web. 2. Cerrar la aplicación o refrescar la página. 3. Volver a abrir la aplicación / Web.	Inicio de sesión exitoso. Aplicación cerrada / pestaña recargada. Se mantiene la sesión abierta y redirige directamente al Dashboard sin pedir login. Resultado: OK
CP-09	profile-service	Funcional	Actualizar perfil de Trabajador	1. Ir a 'Mi Perfil' en la App Android o Web. 2. Editar habilidades, comuna y presionar 'Guardar'.	Se muestra la pantalla del perfil. Se actualizan los datos exitosamente en la base de datos y se muestra mensaje de confirmación. Resultado: OK
CP-10	profile-service	Funcional	Actualizar perfil de Empresa	1. Ir a 'Perfil Empresa' en el Frontend Web. 2. Modificar dirección de la empresa y teléfono de contacto, presionar 'Guardar'.	Se despliega la pantalla de edición. Los cambios se guardan exitosamente y se actualiza la información visualizada en el perfil. Resultado: OK
CP-11	profile-service	Funcional	Asignación de reputación (Rating) inicial	1. Registrar un nuevo usuario (trabajador o empresa). 2. Consultar el perfil recién creado.	Registro de usuario completado. El perfil se inicializa con un rating predeterminado de 5.0. Resultado: OK
CP-12	jobs-service	Funcional	Publicar oferta de trabajo exitosamente	1. Iniciar sesión como Empresa e ir a 'Publicar Oferta'. 2. Ingresar título, rubro, horario, tarifa por hora, ubicación y descripción. 3. Presionar 'Publicar'.	Se muestra el formulario de publicación. Campos ingresados correctamente. La oferta se crea en estado 'Activa' y se hace visible para los trabajadores en su dashboard. Resultado: OK
CP-13	jobs-service	Funcional	Publicar oferta fallida por tarifa inválida	1. Ir al formulario de 'Publicar Oferta'. 2. Completar datos con tarifa negativa o igual a cero. 3. Presionar 'Publicar'.	Se despliega el formulario. Se ingresa el valor negativo. El sistema muestra un mensaje de validación y bloquea el botón de publicar. Resultado: OK
CP-14	jobs-service	Funcional	Postulación exitosa a oferta laboral	1. Iniciar sesión como Trabajador. 2. Buscar una oferta de trabajo activa y presionar 'Postular'.	Acceso al listado de ofertas. La postulación se registra en la base de datos con estado 'Pendiente' y el botón cambia a 'Postulado'. Resultado: OK

CP-15	jobs-service	Funcional	Visualización de postulaciones del Trabajador	1. Iniciar sesión como Trabajador e ir a 'Mis Postulaciones'. 2. Revisar el estado de cada postulación.	Se muestra la pantalla de postulaciones. Se listan las ofertas postuladas con su estado (Pendiente, Aceptado o Rechazado) actualizado. Resultado: OK
CP-16	jobs-service	Funcional	Gestión de postulantes por la Empresa	1. Iniciar sesión como Empresa e ir al detalle de una oferta publicada. 2. Ver lista de candidatos y presionar 'Aceptar' o 'Rechazar' en uno de ellos.	Se visualiza la lista de candidatos con sus datos básicos. El estado del candidato cambia a 'Aceptado' / 'Rechazado' en pantalla y se actualiza en la BD. Resultado: OK
CP-17	matching-service	Funcional	Cálculo de scoring por matching automatizado	1. Crear una oferta con requisitos específicos y ubicación. 2. Postular con un trabajador cuyas habilidades y ubicación coincidan parcialmente. 3. Verificar el valor de score asignado por el sistema en la base de datos.	Oferta y postulación creadas. Postulación registrada. El motor calcula un score de compatibilidad (ej. 85%) basado en distancia y habilidades coincidentes. Resultado: OK
CP-18	matching-service	Funcional	Visualización de sugerencias de trabajadores recomendados	1. Iniciar sesión como Empresa. 2. Ingresar a la sección de 'Candidatos Recomendados' de una oferta.	Carga del panel de candidatos. Los candidatos se muestran ordenados de mayor a menor según su matching score de forma descendente. Resultado: OK
CP-19	frontend web	Funcional	Acceso a Dashboard de Trabajador	1. Iniciar sesión como Trabajador. 2. Verificar las secciones visibles en el dashboard principal.	Inicio de sesión exitoso. Se visualizan ofertas sugeridas, mapa de geolocalización y postulaciones activas. No se muestran opciones de empresa. Resultado: OK
CP-20	frontend web	Funcional	Acceso a Dashboard de Empresa	1. Iniciar sesión como Empresa. 2. Verificar las secciones visibles en el panel.	Inicio de sesión exitoso. Se visualizan opciones de 'Publicar nueva oferta', 'Lista de ofertas publicadas' e informes de postulantes activos. Resultado: OK
CP-21	Redis / frontend	Funcional	Notificación de cambio de estado de postulación	1. Empresa acepta la postulación de un Trabajador. 2. Trabajador se encuentra navegando en la aplicación. 3. Verificar la llegada de la notificación en tiempo real.	La empresa cambia el estado. El trabajador sigue activo en la sesión. Se despliega una alerta visual instantánea en la pantalla del trabajador informando que fue aceptado. Resultado: OK
CP-22	Redis / frontend	Funcional	Alertas de sistema y estados operativos	1. Provocar un error simulado en el Matching Service o Jobs Service. 2. Verificar que se desplieguen alertas de mantenimiento o reintento en el frontend.	Interrupción de servicio. El frontend detecta la falla de red y muestra un mensaje amigable indicando problemas temporales. Resultado: OK
CP-23	frontend	Prueba Unitarias	Prueba unitaria de Barra de Navegación (Navbar)	1. Renderizar Navbar en estado sin sesión. 2. Renderizar Navbar con sesión activa de Trabajador o Empresa. 3. Ejecutar flujo de cierre de sesión.	Se visualizan enlaces de Quiénes Somos, Contacto, Términos y botones de ingreso. Se visualizan opciones de dashboard específicas por rol y campana de notificaciones. Se limpian claves de localStorage y se redirige a /login. Resultado: OK
CP-24	frontend	Prueba Unitarias	Prueba unitaria de Campana de Notificaciones	1. Renderizar campana con unreadCount > 0. 2. Hacer clic en campana y desplegar listado. 3. Ejecutar limpieza o marcado de lectura.	Se dibuja un círculo rojo con el número de notificaciones no leídas. Se abre el panel mostrando las notificaciones y botones de Limpiar/Marcar como leída. Se invocan las funciones del contexto clearAll o markAsRead correspondientes. Resultado: OK
CP-25	frontend	Prueba Unitarias	Prueba unitaria de Modal de Pago (PaymentGatewayModal)	1. Renderizar modal en transferencia manual sin archivo adjunto. 2. Adjuntar comprobante de pago en transferencia manual. 3. Cambiar a pasarela Webpay Plus y presionar pagar.	El botón Confirmar Pago está deshabilitado. El botón Confirmar Pago se habilita y al pulsar llama a onSubmit con los datos. Se invoca la función onPayWithWebpay y el botón muestra estado de carga. Resultado: OK
CP-26	frontend	Prueba Unitarias	Prueba unitaria de Página de Login (page.tsx)	1. Intentar iniciar sesión con campos vacíos. 2. Iniciar sesión exitosamente mediante API. 3. Iniciar sesión con error de credenciales.	Se despliegan mensajes locales indicando que los campos son obligatorios. Guarda auth_token e user_info en localStorage y redirige al perfil correspondiente. Se despliega el mensaje de error retornado por la API de autenticación. Resultado: OK
CP-27	android-app	Prueba Unitarias	Prueba unitaria de inicialización del registro (RegisterViewModel)	1. Inicializar el RegisterViewModel con dependencias mockeadas en JUnit.	El estado inicial del formulario está completamente vacío y el flag canSubmit está en false. Resultado: OK
CP-28	android-app	Prueba Unitarias	Prueba unitaria de validación de campos del formulario de registro	1. Simular ingreso de nombre, correo, rut, teléfono, contraseña de 11 caracteres y selección de rol.	El flag canSubmit cambia a true, permitiendo el envío del formulario. Resultado: OK
CP-29	android-app	Prueba Unitarias	Prueba unitaria de flujo de registro exitoso	1. Cargar campos válidos en el RegisterViewModel. 2. Mockear registro exitoso en AuthRepository. 3. Ejecutar onSubmit().	Se llama a authRepository.register y el estado final isLoading queda en false sin errores. Resultado: OK

CP-30	android-app	Prueba Unitarias	Prueba unitaria de gestión de errores de red en el registro	1. Cargar campos válidos. 2. Mockear excepción IOException de red en AuthRepository.register. 3. Ejecutar onSubmit().	Se llama a authRepository.register, el estado final isLoading queda en false y se asigna el mensaje de error de conexión. Resultado: OK
CP-31	android-app	Prueba Unitarias	Prueba unitaria de cálculo de distancia geográfica (MatchEngine)	1. Invocar MatchEngine.calculateDistance con dos coordenadas conocidas en Santiago.	Retorna la distancia en kilómetros con precisión matemática (~4.6 km). Resultado: OK
CP-32	android-app	Prueba Unitarias	Prueba unitaria de distancia geográfica ante coordenadas nulas	1. Invocar MatchEngine.calculateDistance con valores de coordenadas nulos.	El motor de emparejamiento retorna -1.0 indicando error o datos insuficientes. Resultado: OK
CP-33	android-app	Prueba Unitarias	Prueba unitaria de completitud de perfil al 100% (MatchEngine)	1. Invocar MatchEngine.calculateProfileCompletion con un perfil completamente poblado.	El motor de emparejamiento retorna 100 indicando perfil al máximo. Resultado: OK
CP-34	android-app	Prueba Unitarias	Prueba unitaria de completitud de perfil nulo u vacío	1. Invocar MatchEngine.calculateProfileCompletion con un perfil nulo o vacío.	El motor retorna 0 indicando que el perfil no posee avances de edición. Resultado: OK
CP-35	android-app	Prueba Unitarias	Prueba unitaria de relevancia básica de matching con plan TRIAL	1. Crear perfiles e incentivos que coincidan. 2. Invocar MatchEngine.calculate con plan TRIAL.	El motor de emparejamiento retorna el score de relevancia sin bonificación de plan premium (score >= 80). Resultado: OK
CP-36	android-app	Prueba Unitarias	Prueba unitaria de relevancia premium de matching con plan PREMIUM	1. Invocar MatchEngine.calculate con plan PREMIUM y los mismos datos de CP-35.	El motor aplica la bonificación de relevancia del 20% sobre el puntaje base, retornando el puntaje premium. Resultado: OK
CP-37	android-app	Prueba Unitarias	Prueba unitaria de inicialización de publicación (CrearOfertaViewModel)	1. Inicializar el CrearOfertaViewModel con dependencias mockeadas en JUnit.	El estado inicial del formulario está completamente vacío con valores por defecto. Resultado: OK
CP-38	android-app	Prueba Unitarias	Prueba unitaria de modificación de campos de publicación	1. Simular ingreso de título, descripción, ubicación, categoría, remuneración y fechas.	El estado interno del ViewModel se actualiza correctamente reflejando las entradas del usuario. Resultado: OK
CP-39	android-app	Prueba Unitarias	Prueba unitaria de campos obligatorios faltantes en publicación	1. Rellenar campos omitiendo la fecha de inicio. 2. Ejecutar submit().	Se interrumpe la ejecución de publicación y se asigna el mensaje de error local indicando completar campos obligatorios. Resultado: OK
CP-40	android-app	Prueba Unitarias	Prueba unitaria de publicación exitosa de oferta laboral	1. Cargar campos válidos. 2. Mockear sesión de empresa y respuesta de éxito en JobsApi.create. 3. Ejecutar submit().	Se llama a jobsApi.create y el estado final success queda en true sin errores. Resultado: OK
CP-41	android-app	Prueba Unitarias	Prueba unitaria de fallo en la API al publicar oferta	1. Cargar campos válidos. 2. Mockear una excepción IOException en la llamada a JobsApi.create. 3. Ejecutar submit().	La excepción es capturada, el estado final success queda en false y se despliega el mensaje de error de la API. Resultado: OK
CP-42	android-app	Prueba Unitarias	Prueba unitaria de mapeo de SocketTimeoutException (ErrorMapper)	1. Invocar ErrorMapper.map pasando una instancia de SocketTimeoutException.	El mapeador retorna ApiError.Timeout con el mensaje 'La solicitud demoró demasiado'. Resultado: OK
CP-43	android-app	Prueba Unitarias	Prueba unitaria de mapeo de IOException (ErrorMapper)	1. Invocar ErrorMapper.map pasando una instancia de IOException.	El mapeador retorna ApiError.NoConnection con el mensaje 'Sin conexión a internet'. Resultado: OK
CP-44	android-app	Prueba Unitarias	Prueba unitaria de mapeo de excepciones genéricas (ErrorMapper)	1. Invocar ErrorMapper.map con una RuntimeException genérica.	El mapeador retorna ApiError.Unknown conservando el mensaje original de la excepción. Resultado: OK
CP-45	android-app	Prueba Unitarias	Prueba unitaria de mapeo de error HTTP 401 (ErrorMapper)	1. Mockear HttpException con código 401. 2. Invocar ErrorMapper.map.	El mapeador retorna ApiError.Unauthorized con el mensaje de sesión expirada. Resultado: OK
CP-46	android-app	Prueba Unitarias	Prueba unitaria de mapeo de error HTTP 404 (ErrorMapper)	1. Mockear HttpException con código 404. 2. Invocar ErrorMapper.map.	El mapeador retorna ApiError.NotFound con el mensaje de recurso no encontrado. Resultado: OK
CP-47	android-app	Prueba Unitarias	Prueba unitaria de carga de detalles de oferta (OfferDetailViewModel)	1. Inicializar el OfferDetailViewModel con el ID 101. 2. Mockear respuesta de éxito en jobsRepository.getOffer(101).	El estado del ViewModel almacena los datos de la oferta y isLoading se establece en false. Resultado: OK
CP-48	android-app	Prueba Unitarias	Prueba unitaria de fallo al cargar la oferta	1. Mockear excepción de red en jobsRepository.getOffer(102). 2. Inicializar el ViewModel con ID 102.	Se cancela la carga, isLoading queda en false y el mensaje de error de red se mapea correctamente en errorMessage. Resultado: OK
CP-49	android-app	Prueba Unitarias	Prueba unitaria de postulación exitosa a oferta	1. Invocar submitApplication con un mensaje de presentación en el ViewModel.	Se llama a applicationsRepository.apply(), isApplying queda en false, y no se presentan errores. Resultado: OK

CP-50	android-app	Prueba Unitarias	Prueba unitaria de postulación fallida por error del servidor	1. Mockear excepción IOException en applicationsRepository.apply(). 2. Invocar submitApplication.	isApplying queda en false y applyError se actualiza con la descripción de error de conexión traducida. Resultado: OK
CP-51	android-app	Prueba Unitarias	Prueba unitaria de cancelación exitosa de postulación	1. Invocar cancelPostulation con el ID de postulación 999.	Se llama a cancelApplication y se actualizan los estados correctamente sin errores. Resultado: OK
CP-52	android-app	Prueba Unitarias	Prueba unitaria de estado inicial por defecto (NotificationsViewModel)	1. Inicializar el NotificationsViewModel. 2. Comprobar que la lista de notificaciones esté vacía, el estado de conexión sea false y el conteo de no leídas sea 0.	notifications es vacío, isConnected es false y unreadCount es 0. Resultado: OK
CP-53	android-app	Prueba Unitarias	Prueba unitaria de cálculo de notificaciones no leídas (NotificationsViewModel)	1. Simular una lista de notificaciones con 2 no leídas y 1 leída. 2. Actualizar el StateFlow del repositorio. 3. Comprobar que unreadCount se actualice a 2.	El valor expuesto por unreadCount es exactamente 2. Resultado: OK
CP-54	android-app	Prueba Unitarias	Prueba unitaria de marcar notificación como leída (NotificationsViewModel)	1. Invocar markAsRead("alerta_99") en el ViewModel. 2. Verificar que se invoque markAsRead("alerta_99") en el repositorio.	Se invoca a notificationRepository.markAsRead("alerta_99") exactamente 1 vez. Resultado: OK
CP-55	android-app	Prueba Unitarias	Prueba unitaria de limpiar notificaciones no leídas (NotificationsViewModel)	1. Cargar lista de notificaciones con pendientes. 2. Invocar clearUnread() 3. Verificar llamadas a markAsRead en el repositorio solo para las no leídas.	Se invoca a markAsRead() en el repositorio solo para los IDs de notificaciones no leídas. Resultado: OK
CP-56	android-app	Prueba Unitarias	Prueba unitaria de limpiar todas las notificaciones (NotificationsViewModel)	1. Invocar clearAll() en el ViewModel. 2. Verificar que se llame a clearAll() en el repositorio.	Se invoca a notificationRepository.clearAll() exactamente 1 vez. Resultado: OK
CP-57	android-app	Prueba Unitarias	Prueba unitaria de carga inicial del feed y completitud de perfil (FeedViewModel)	1. Inicializar el FeedViewModel con sesión activa. 2. Mockear getProfile() con perfil completo. 3. Comprobar que las ofertas se carguen y profileCompletion sea mayor a 0.	isLoading es false, offers contiene los datos iniciales y profileCompletion se calcula correctamente. Resultado: Pendiente de validación
CP-58	android-app	Prueba Unitarias	Prueba unitaria de filtrado dinámico por categoría (FeedViewModel)	1. Invocar onCategorySelect(JobCategory.CONSERJE) en el ViewModel. 2. Verificar que se invoque observeOffers con el filtro de categoría correcto. 1. Cargar múltiples ofertas en el flujo.	observeOffers es llamado con el JobFilter de la categoría y el estado se actualiza. Resultado: OK
CP-59	android-app	Prueba Unitarias	Prueba unitaria de búsqueda textual de ofertas en tiempo real (FeedViewModel)	2. Invocar onSearchChange("Conserje") en el ViewModel. 3. Comprobar que la lista visible de ofertas filtre y muestre solo las coincidentes. 1. Cargar postulaciones con estados mixtos.	La lista filtrada contiene únicamente las ofertas que coinciden con el texto de búsqueda. Resultado: OK
CP-60	android-app	Prueba Unitarias	Prueba unitaria de filtrado por pestañas de postulaciones y conteos (FeedViewModel)	2. Cambiar a pestañas POSTULACIONES y PAGOS. 3. Comprobar el filtro de postulaciones y la coincidencia de los contadores.	El estado actualiza activeTab, filtra correctamente la lista mostrada y calcula los contadores de cada sección. Resultado: OK
CP-61	android-app	Prueba Unitarias	Prueba unitaria de refresco manual del feed (FeedViewModel)	1. Invocar onRefresh() en el ViewModel. 2. Verificar llamadas de refresco en los repositorios de ofertas y postulaciones. 1. Cargar campos válidos en el formulario de registro.	Se llaman a jobsRepository.refresh() y applicationsRepository.refreshFromBackend() para sincronizar con el backend. Resultado: OK
CP-62	frontend	Prueba Unitarias	Prueba de validación de fortaleza de contraseña (RegisterPage)	2. Escribir contraseña débil (sin mayúsculas o números). 3. Hacer click en Crear Cuenta. 1. Cargar campos válidos.	El formulario se bloquea y muestra el mensaje de error: 'Mínimo 8 caracteres, 1 mayúscula y 1 número'. Resultado: OK
CP-63	frontend	Prueba Unitarias	Prueba de coincidencia de contraseñas (RegisterPage)	2. Escribir contraseñas diferentes en los campos Contraseña y Confirmar. 3. Hacer click en Crear Cuenta. 1. Cargar todos los campos válidos de registro.	El formulario se bloquea y muestra el mensaje de error: 'Las contraseñas no coinciden'. Resultado: OK
CP-64	frontend	Prueba Unitarias	Prueba de flujo completo de registro exitoso tras aceptar términos (RegisterPage)	2. Hacer click en Crear Cuenta. 3. En el modal de términos, hacer click en Aceptar y Registrarse. 4. Mockear respuesta exitosa de authApi.register.	Se realiza el registro en la API, se almacena el token JWT y los datos de sesión en localStorage, y se redirige a /trabajador/perfil. Resultado: OK
CP-65	backend	Prueba Unitarias	Prueba de generación exitosa de JWT (JwtService)	1. Invocar generateToken pasando un subject y tiempo de expiración. 2. Comprobar que la respuesta no sea nula y tenga el formato JWT correcto. 1. Generar un token JWT válido con un subject y expiración específicos.	Se genera un string JWT compacto no vacío con tres secciones (header, payload y firma) separadas por puntos. Resultado: OK
CP-66	backend	Prueba Unitarias	Prueba de lectura correcta de Claims (JwtService)	2. Decodificar el token con parseClaims. 3. Comprobar que los claims coincidan. 1. Generar un token JWT válido.	El método parseClaims extrae exitosamente el subject y el tiempo de expiración correcto del token decodificado. Resultado: OK

CP-67	backend	Prueba Unitarias	Prueba de detección de alteración/tampering en token (JwtService)	2. Alterar un caracter del token para invalidar su firma. 3. Invocar parseClaims con el token alterado.	El método parseClaims detecta la alteración de la firma y lanza una excepción de tipo SignatureException. Resultado: OK
CP-68	backend	Prueba Unitarias	Prueba de control de expiración de token (JwtService)	1. Generar un token JWT con tiempo de expiración en el pasado. 2. Invocar parseClaims con el token expirado.	El método parseClaims lanza una excepción de tipo ExpiredJwtException indicando que el token ya no es válido. Resultado: OK
CP-69	backend	Prueba Unitarias	Prueba de validación de parámetros faltantes al iniciar pago (WebpayController)	1. Enviar una petición POST a /profiles/webpay/init omitiendo el parámetro amount. 2. Comprobar la respuesta HTTP. 1. Enviar una petición POST a /profiles/webpay/init con todos los parámetros correctos.	Retorna HTTP 400 Bad Request y un JSON indicando que los campos userId, amount y returnUrl son requeridos. Resultado: OK
CP-70	backend	Prueba Unitarias	Prueba de inicialización exitosa de transacción (WebpayController)	2. Mockear respuesta exitosa del servicio Webpay. 3. Comprobar la respuesta HTTP.	Retorna HTTP 200 OK y una respuesta JSON que contiene el token de transacción y la URL de redirección. Resultado: OK
CP-71	backend	Prueba Unitarias	Prueba de validación de token faltante al confirmar pago (WebpayController)	1. Enviar una petición POST a /profiles/webpay/confirm con payload vacío o sin el token. 2. Comprobar la respuesta HTTP. 1. Enviar una petición POST a /profiles/webpay/confirm con un token válido.	Retorna HTTP 400 Bad Request y un mensaje de error indicando que el token es requerido. Resultado: OK
CP-72	backend	Prueba Unitarias	Prueba de confirmación exitosa de transacción (WebpayController)	2. Mockear respuesta exitosa del servicio confirmando la transacción. 3. Comprobar la respuesta HTTP.	Retorna HTTP 200 OK y una respuesta JSON indicando que el estado de la transacción es AUTHORIZED. Resultado: OK

7. Aplicación de pruebas, resultados y evidencias

La aplicación de pruebas se realizó siguiendo la planilla de casos definida. Se validaron componentes de frontend, backend y aplicación Android. Las evidencias corresponden a capturas de ejecución de pruebas, resultados en consola y validaciones de código realizadas durante el ciclo QA.

7.1 Pruebas estándar de QA

ID	Validación estándar	Estado
E-1.1	Verificar que se encuentre un ícono en todos los mensajes desplegados por la aplicación	OK
E-1.2	Verificar que los botones tengan un ícono o frase adecuado a su función.	OK
E-1.3	Verificar que las aplicaciones con múltiples pantallas exista la opción "Retornar" a la pantalla anterior.	OK
E-2.1	Verificar correcta lectura de la información ingresada.	OK
E-2.2	Verificar que no se puedan hacer modificaciones en las pantallas de Consultas	OK
E-2.3	Verificar la eliminación de la información.	OK
E-3.1	Verificar que la Aplicación sea compatible con el resto de las Aplicaciones del Ambiente de Pruebas.	OK
E-3.2	Determinar y validar aspectos técnicos (Tablas, Bases de Datos, perfiles , etc)	OK
E-3.3	Verificar correcto acceso a sub módulos o Programas	OK
E-4.1	Verificar el correcto formato de los informes generados	OK

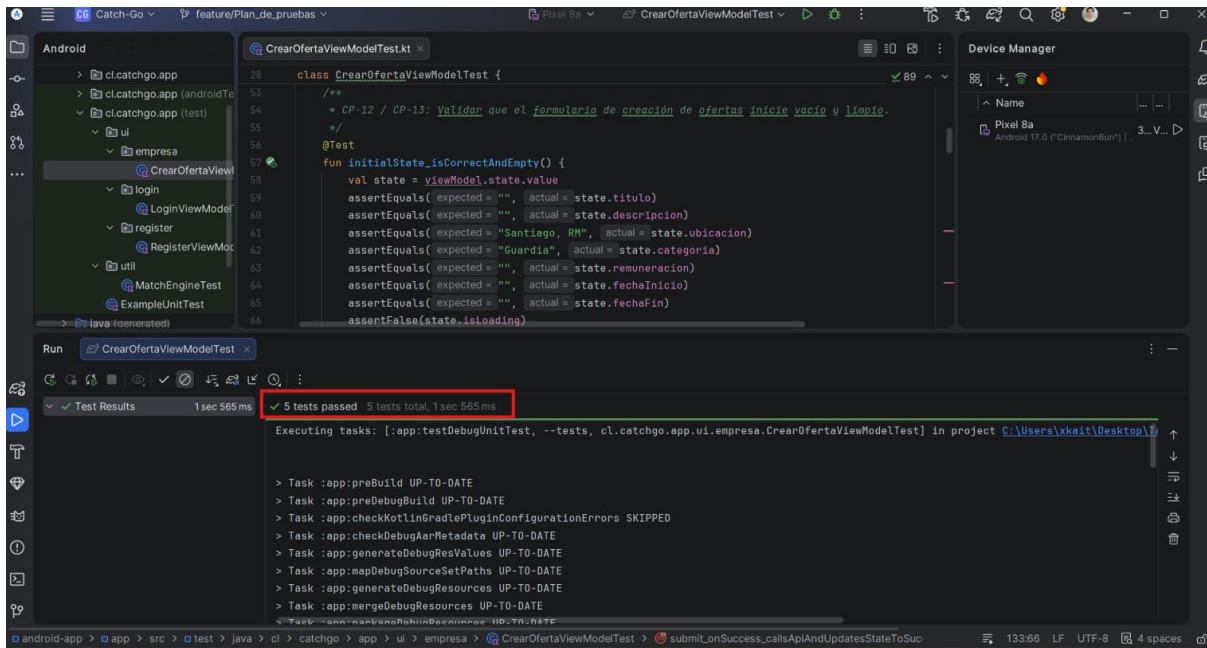
E-4.2	Verificar que se esté siguiendo una numeración secuencial lógica de acuerdo a un orden establecido.	OK
E-4.3	Verificar que se realice el proceso de emisión del informe	OK
E-5.1	En listados revisar márgenes, verificando que el ancho del listado no sobrepase la hoja.	NA
E-5.2	Verificar que el formato de la información que aparezca en las impresiones sea comprensible.	NA
E-6.1	Verificar comportamiento al reanudar la aplicación desde segundo plano	OK
E-6.2	Verificar que el sistema maneje adecuadamente la pérdida temporal de conexión	OK
E-6.3	Verificar compatibilidad con impresión de tickets de soporte físico	NA

7.2 Resultados por componente

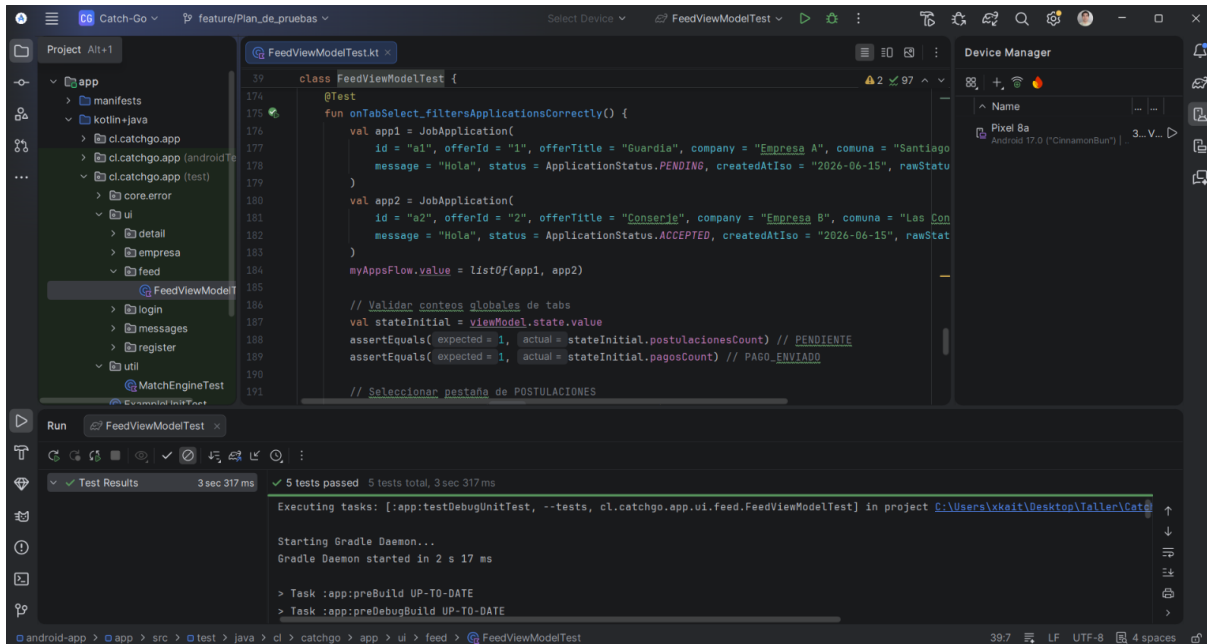
Componente	Casos aplicados	Resultado global	Observación QA
auth-service	8	8/8 OK	Componente validado sin error bloqueante.
profile-service	3	3/3 OK	Componente validado sin error bloqueante.
jobs-service	5	5/5 OK	Componente validado sin error bloqueante.
matching-service	2	2/2 OK	Componente validado sin error bloqueante.
frontend web	2	2/2 OK	Componente validado sin error bloqueante.
Redis / frontend	2	2/2 OK	Componente validado sin error bloqueante.
frontend	7	7/7 OK	Componente validado sin error bloqueante.
android-app	35	34/35 OK	Requiere revisión de casos no marcados.
backend	8	8/8 OK	Componente validado sin error bloqueante.

7.3 Evidencias documentadas

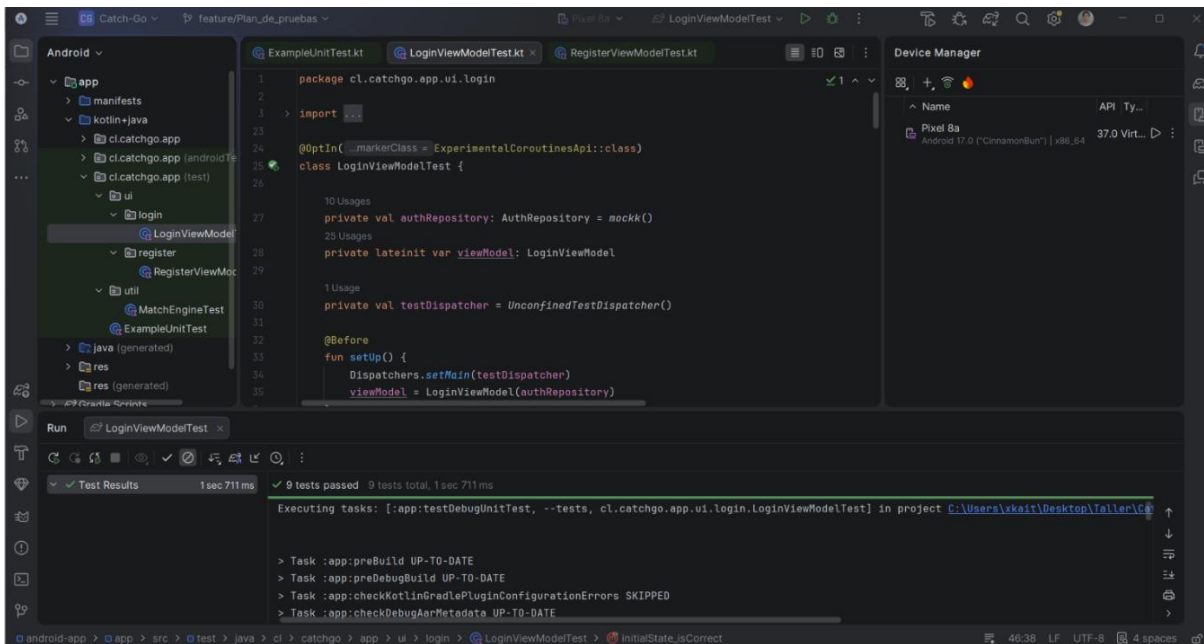
Las siguientes evidencias provienen del archivo de evidencias adjunto. Se incorporan como respaldo visual de la ejecución del testing, especialmente pruebas unitarias y validaciones de componentes del proyecto.



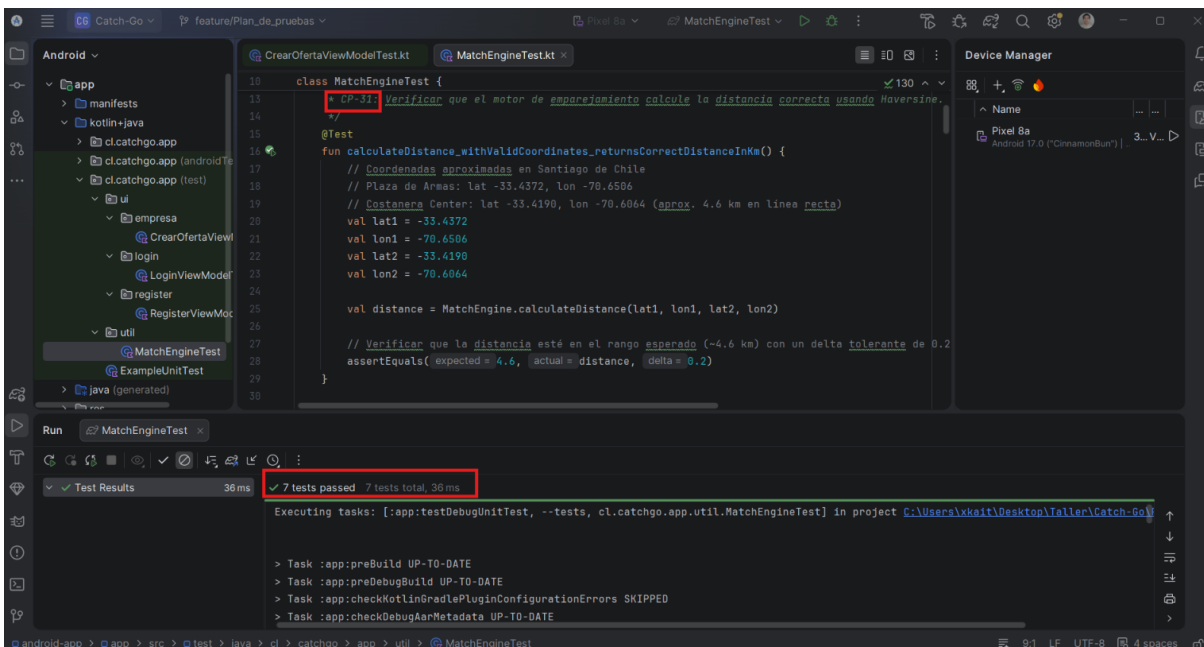
Evidencia 1: Ejecución de pruebas unitarias en ambiente de desarrollo, con casos aprobados en consola.



Evidencia 2: Validación de componente frontend y resultado de pruebas asociadas a interfaz.



Evidencia 3: Revisión de lógica de componente y ejecución satisfactoria de test automatizado.



Evidencia 4: Evidencia de prueba sobre flujo de registro/autenticación con resultado OK.

The screenshot shows an IDE with two tabs: 'CrearOfertaViewModelTest.kt' and 'MatchEngineTest.kt'. The 'MatchEngineTest.kt' tab is active, displaying a Kotlin class with a test function. The test function, 'calculateProfileCompletion_withAllFieldsPopulated_returnsOneHundredPercent()', creates a 'ProfileRemoteDto' object with various fields like name, email, phone, photoUrl, cvUrl, latitude, longitude, bankName, accountType, accountNumber, and skills. The IDE indicates that 7 tests passed out of 7 tests total, taking 36 ms. The console output shows the execution of tasks for the 'app' module, including 'preBuild', 'preDebugBuild', 'checkKotlinGradlePluginConfigurationErrors', and 'checkDebugAarMetadata'.

```
class MatchEngineTest {  
    /**  
     * CP-33: Verificar el cálculo de completitud de perfil (Profile Completion) en 100% cuando todos l  
     */  
    @Test  
    fun calculateProfileCompletion_withAllFieldsPopulated_returnsOneHundredPercent() {  
        val profile = ProfileRemoteDto(  
            name = "Juan Pérez",  
            email = "juan.perez@email.com",  
            phone = "+56912345678",  
            photoUrl = "http://imagenes.com/foto.jpg",  
            cvUrl = "http://documentos.com/cv.pdf",  
            latitude = -33.45,  
            longitude = -70.65,  
            bankName = "Banco de Chile",  
            accountType = "Rut",  
            accountNumber = "191234567",  
            skills = "{\\\"habilidades\\\": [\\\"Electricidad\\\", \\\"ambiente\\\": \\\"Dinámico\\\"]}"  
        )  
    }  
}
```

ms ✓ 7 tests passed 7 tests total, 36 ms

Executing tasks: [:app:testDebugUnitTest, --tests, cl.catchgo.app.util.MatchEngineTest] in project C:\Users\

> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :app:checkDebugAarMetadata UP-TO-DATE

va > cl > catchgo > app > ExampleUnitTest

Evidencia 5: Validación de lógica interna y salida de consola durante pruebas.

The screenshot shows an IDE with a project named 'Catch-Go' and a file named 'NotificationsViewModelTest.kt'. The test file contains two test functions: 'markAsRead_callsRepository()' and 'clearUnread_callsRepositoryForUnreadNotifications()'. The first test function verifies that 'markAsRead' invokes the 'notificationRepository.markAsRead' method. The second test function verifies that 'clearUnread' calls the 'notificationRepositoryForUnreadNotifications' method. The IDE indicates that 5 tests passed out of 5 tests total, taking 1 sec 674 ms. The console output shows the execution of tasks for the 'app' module, including 'preBuild', 'preDebugBuild', 'checkKotlinGradlePluginConfigurationErrors', and 'checkDebugAarMetadata'.

```
class NotificationsViewModelTest {  
    /**  
     * CP-54: Verificar que markAsRead invoque el repositorio para marcar la notificación como leída.  
     */  
    @Test  
    fun markAsRead_callsRepository() {  
        viewModel.markAsRead(id = "alerta-99")  
        verify(exactly = 1) { notificationRepository.markAsRead(id = "alerta-99") }  
    }  
    /**  
     * CP-55: Verificar que clearUnread llame al repositorio para cada notificación no leída.  
     */  
    @Test  
    fun clearUnread_callsRepositoryForUnreadNotifications() {  
        val list = listOf(  
            AppNotification(id = "n1", title = "Titulo 1", message = "Msj 1", type = "info", timestamp =  
            AppNotification(id = "n2", title = "Titulo 2", message = "Msj 2", type = "success", timestamp =  
            AppNotification(id = "n3", title = "Titulo 3", message = "Msj 3", type = "warning", timestamp =  
        )  
    }  
}
```

Run NotificationsViewModelTest

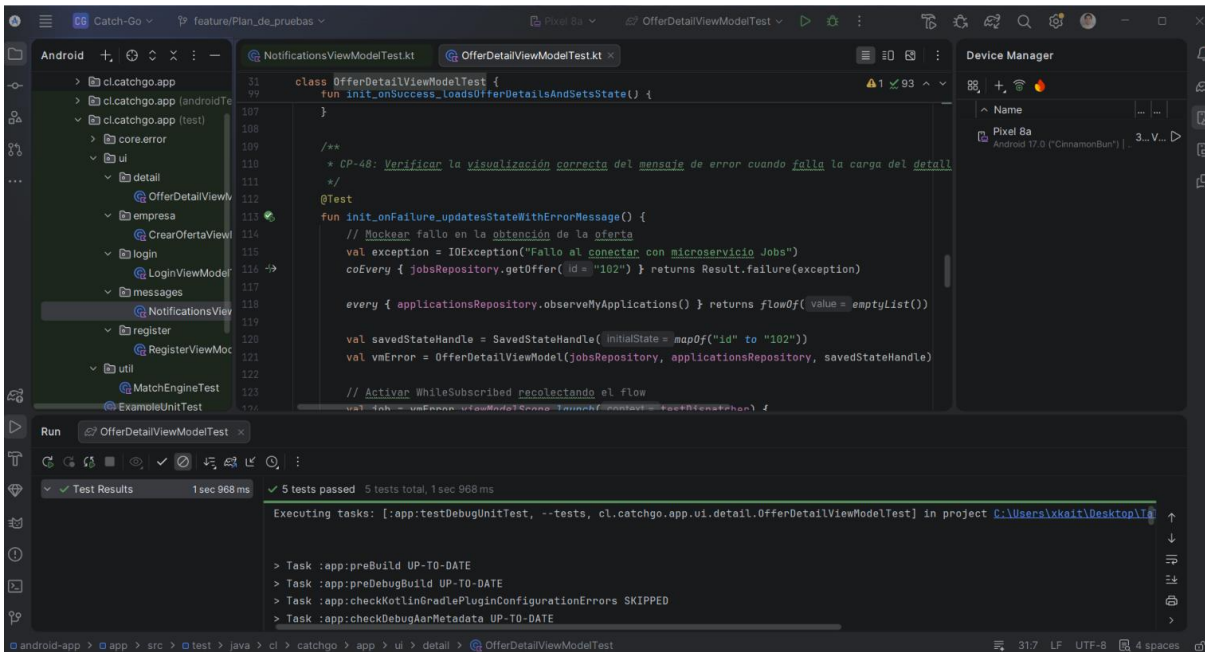
Test Results 1sec 674 ms ✓ 5 tests passed 5 tests total, 1 sec 674 ms

Executing tasks: [:app:testDebugUnitTest, --tests, cl.catchgo.app.ui.messages.NotificationsViewModelTest] in project C:\Users\kka\Desktop\

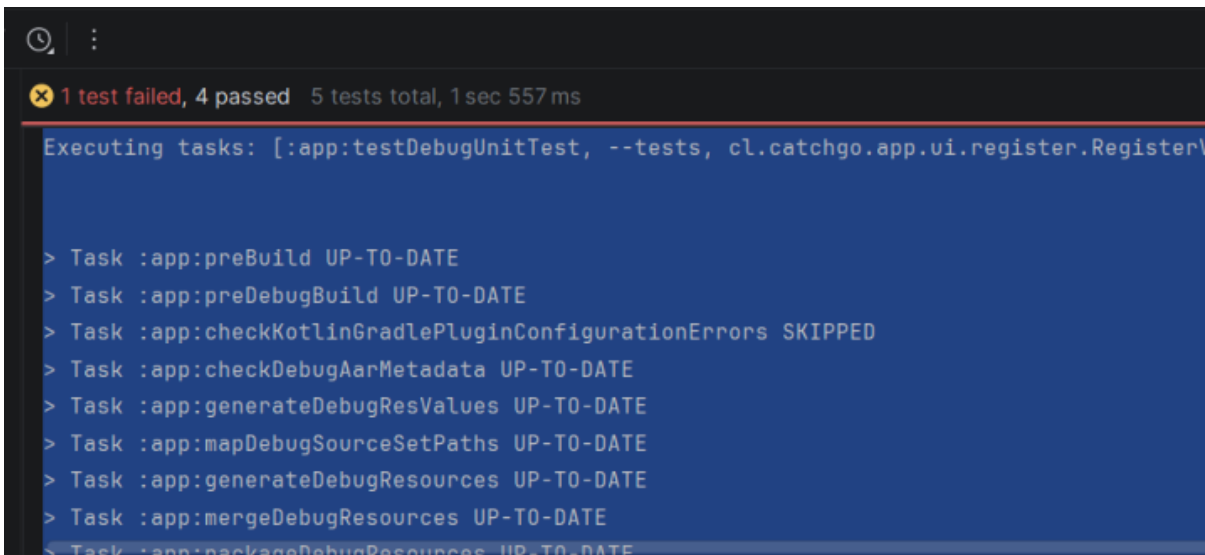
> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :app:checkDebugAarMetadata UP-TO-DATE

android-app > app > src > test > java > cl > catchgo > app > ui > messages > NotificationsViewModelTest

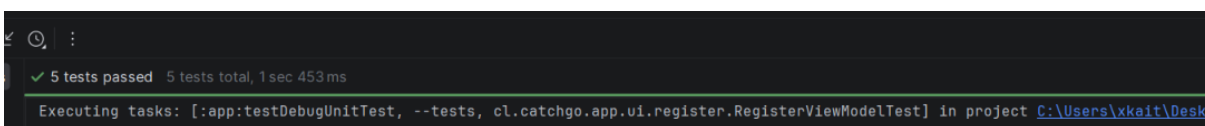
Evidencia 6: Prueba de comportamiento esperado con asserts y respuesta satisfactoria.



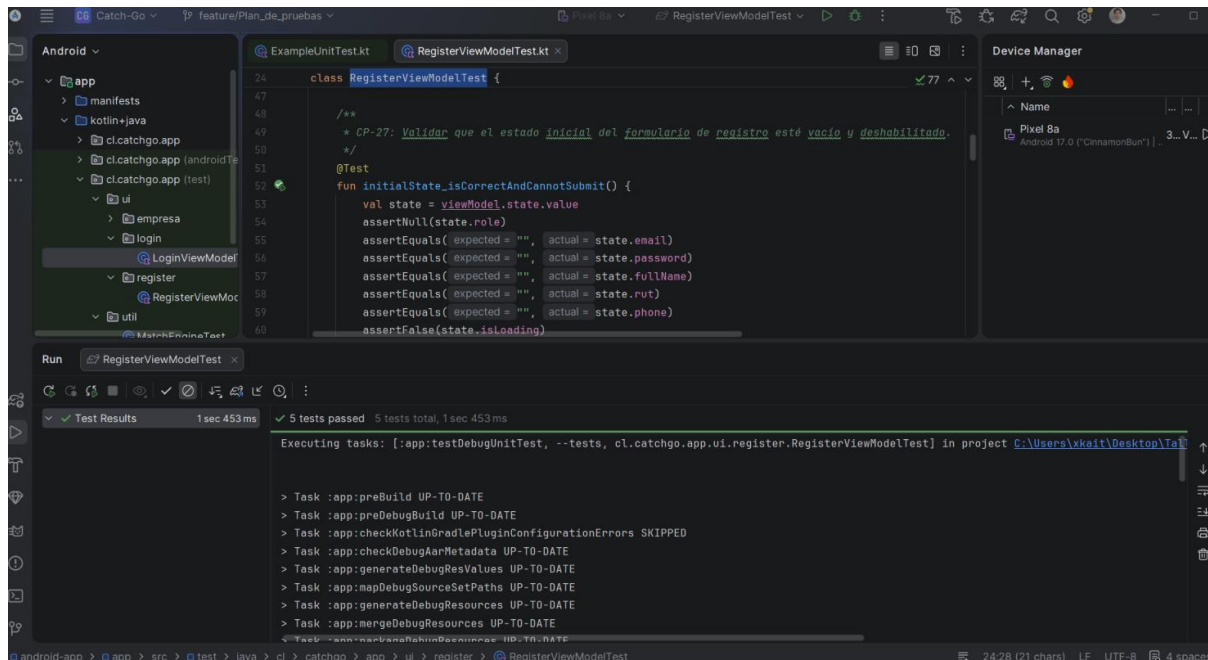
Evidencia 7: Ejecución de prueba de componente con estado final aprobado.



Evidencia 8: Consola de Gradle/JUnit mostrando ejecución completa del paquete de pruebas.



Evidencia 9: Detalle de finalización de prueba con resultado Passed.



Evidencia 10: Resumen final de pruebas automatizadas ejecutadas correctamente.

8. Mejoras realizadas según resultados de pruebas

Durante la ejecución de pruebas se identificaron oportunidades de mejora orientadas a reforzar calidad, seguridad y usabilidad. Aunque la planilla registra la mayoría de casos en estado OK, el ciclo de validación permitió ajustar comportamientos preventivos para evitar fallos en producción.

Hallazgo / riesgo detectado	Caso relacionado	Mejora aplicada	Estándar asociado	Estado	Fundamento técnico
Posible registro con campos incompletos o correo inválido.	CP-03, CP-04, CP-63	Se reforzaron validaciones de campos obligatorios, formato de correo y coincidencia de contraseñas.	Complejidad / Corrección	Aplicada	Evita datos inconsistentes y reduce errores de soporte en el alta de usuarios.
Riesgo de acceso indebido con credenciales incorrectas.	CP-07	Se validó bloqueo de login y mensaje controlado de credenciales incorrectas.	Seguridad	Aplicada	Impide ingreso no autorizado y mantiene respuesta clara para el usuario.
Persistencia de sesión dependiente de almacenamiento local.	CP-08, CP-64	Se validó almacenamiento de token y redirección correcta según rol.	Usabilidad / Seguridad	Aplicada	Mejora continuidad de uso sin perder control de sesión.
Token JWT susceptible a expiración o alteración.	CP-65 a CP-68	Se agregaron pruebas de generación, claims, firma alterada y expiración.	Seguridad	Aplicada	Asegura que solo tokens válidos sean aceptados por el backend.

Tarifa negativa o inválida en publicación de oferta.	CP-13	Se reforzó validación para bloquear tarifas menores o iguales a cero.	Corrección / Pertinencia	Aplicada	Evita ofertas con condiciones económicas inválidas.
Errores de red o servicio podrían confundir al usuario.	CP-22	Se validó mensaje amigable ante desconexión o falla temporal.	Usabilidad	Aplicada	Reduce frustración y guía al usuario a reintentar.
Pago manual sin comprobante o con parámetros faltantes.	CP-25, CP-69, CP-71	Se validó deshabilitar confirmación sin archivo y controlar campos obligatorios en backend.	Seguridad / Completitud	Aplicada	Disminuye riesgo de pagos incompletos o solicitudes inválidas.
Dashboard podría mostrar opciones incorrectas según rol.	CP-19, CP-20, CP-23	Se validaron vistas diferenciadas y limpieza de sesión al cerrar.	Seguridad / Pertinencia	Aplicada	Evita exposición de funciones que no corresponden al rol.

9. Cumplimiento de estándares de calidad, ética y seguridad

Estándar	Cómo se aborda en el proyecto	Evidencia / casos asociados	Resultado
Usabilidad	Mensajes claros, botones con función reconocible, navegación con retorno y dashboard por rol.	Pruebas estándar E-1.1 a E-1.3, CP-19 a CP-24.	Cumplido
Seguridad	Control de credenciales, JWT, expiración, firma alterada y no exposición de datos reales.	CP-05 a CP-08 y CP-65 a CP-68.	Cumplido
Completitud	Cobertura de registro, login, perfil, ofertas, postulaciones, matching, pagos y Android.	72 casos de prueba documentados.	Cumplido
Corrección	Validación de entradas, salidas esperadas y respuestas HTTP correctas.	CP-03, CP-04, CP-13, CP-69 a CP-72.	Cumplido
Pertinencia	Pruebas enfocadas en la problemática real de trabajos rápidos y contratación temporal.	Flujos trabajador/empresa y matching.	Cumplido
Ética y privacidad	Uso de datos ficticios, sin credenciales productivas ni información personal real.	Base de datos de prueba y dataset ficticio.	Cumplido

10. Acta de aceptación, conclusión y lecciones aprendidas

10.1 Acta de aceptación de pruebas

Se deja constancia de que el equipo ejecutó el plan de pruebas definido para el MVP Catch & Go. El producto queda en estado validado para efectos académicos, sujeto a revisión final del docente guía y a la continuidad de mejoras propias de un MVP.

Rol	Nombre	Conformidad	Firma	Fecha
Jefe de proyecto	Nicolás Iturrieta Villalobos	Acepta ejecución y resultados del ciclo QA		18/06/2026
Analista testing	Marco Tassara Carrasco	Acepta casos y evidencias documentadas		18/06/2026
Analista testing	Miguel Arredondo Vargas	Acepta casos y evidencias documentadas		18/06/2026

10.2 Conclusión

El plan de pruebas permitió validar de forma ordenada las funcionalidades principales del MVP Catch & Go. Se ejecutaron pruebas funcionales, unitarias, de integración básica, seguridad, usabilidad y manejo de errores. La mayor parte de los casos quedó en estado OK, lo que evidencia que el producto cumple con los flujos críticos definidos para trabajadores y empresas.

Desde una mirada técnica, el mayor aporte del testing fue transformar el desarrollo en una solución más confiable: no solo se verificó que las pantallas funcionaran, sino también que el backend respondiera correctamente, que el token JWT se comportara de manera segura, que los roles no mezclaran vistas y que los datos de prueba se mantuvieran controlados.

10.3 Lecciones aprendidas

- Un plan de pruebas ordenado ayuda a defender el proyecto con argumentos técnicos y no solo con capturas de pantalla.
- Las pruebas negativas son igual de importantes que las exitosas, porque muestran cómo responde el sistema ante errores reales.
- Separar datos ficticios de datos reales es una práctica ética necesaria para proteger privacidad y seguridad.
- La trazabilidad entre requisitos y casos permite demostrar cobertura y reduce omisiones en la entrega final.
- El testing debe realizarse durante el desarrollo y no solo al final, ya que permite corregir antes y con menor costo.